Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №6
По дисциплине «Современные платформы программирования»

Выполнил:
Студент 3 курса
Группы ПО-12
Авдей Д.Б.
Проверил:
Кулик А.Д.

Брест 2026

Цель: освоить приемы тестирования кода на примере использования пакета pytest.

Вариант №1

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)

Код:

```
# 3. Фикстура для пустой корзины
@pytest.fixture
def empty_cart():
    return Cart()


# 1. Добавление товара
def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1
    assert empty_cart.items[0]["name"] == "Apple"
    assert empty_cart.items[0]["price"] == 10.0


#  Тест отрицательной цены
def test_add_item_negative_price(empty_cart):
    with pytest.raises(ValueError):
        empty_cart.add_item("Banana", -5)


#  Тест подсчета общей стоимости
def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.0)
    assert empty_cart.total() == 15.0


# 2. Тест apply_discount с параметризацией
@pytest.mark.parametrize(
    "discount, expected",
    [
        (0, 20.0),
        (50, 10.0),
        (100, 0.0),
    ],
)
def test_apply_discount(empty_cart, discount, expected):
    empty_cart.add_item("Apple", 20.0)
    empty_cart.apply_discount(discount)
    assert empty_cart.total() == expected


@pytest.mark.parametrize("discount", [-10, 150])
def test_apply_discount_invalid(empty_cart, discount):
    empty_cart.add_item("Apple", 10.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(discount)
```

```
def test_log_purchase():
    with patch("shopping.requests.post") as mock_post:
        item = {"name": "Apple", "price": 10.0}
        log_purchase(item)
        mock_post.assert_called_once_with("https://example.com/log",
json=item)


# 5. Тест apply_coupon


def test_apply_coupon_save10(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    apply_coupon(empty_cart, "SAVE10")
    assert empty_cart.total() == 90.0


def test_apply_coupon_half(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    apply_coupon(empty_cart, "HALF")
    assert empty_cart.total() == 50.0


def test_apply_coupon_invalid(empty_cart):
    empty_cart.add_item("Apple", 100.0)
    with pytest.raises(ValueError):
        apply_coupon(empty_cart, "INVALID")


def test_apply_coupon_monkeypatch(empty_cart, monkeypatch):
    empty_cart.add_item("Apple", 100.0)
    fake_coupons = {"FAKE50": 50}

    monkeypatch.setattr(shopping, "coupons", fake_coupons)

    apply_coupon(empty_cart, "FAKE50")
    assert empty_cart.total() == 50.0
```

Задание 2: Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Код:

```
@pytest.mark.parametrize(
    "data, expected",
    [
        ([], False),  # пустой список
        ([1], True),  # один элемент
        ([1, 1, 1], True),  # все одинаковые
        ([1, 2, 1], False),  # разные элементы
```

```
        (["a", "a", "a"], True),  # строки одинаковые
        (["a", "b"], False),  # строки разные
    ],
)
def test_all_elements_equal(data, expected):
    assert all_elements_equal(data) == expected


def test_find_two_sum_simple():
    arr = [2, 7, 11, 15]
    target = 9
    assert find_two_sum(arr, target) == [0, 1]


def test_find_two_sum_multiple_pairs():
    arr = [1, 2, 3, 4, 5]
    target = 6
    assert find_two_sum(arr, target) == [0, 4]


def test_find_two_sum_no_solution():
    arr = [1, 2, 3]
    target = 10
    assert find_two_sum(arr, target) is None


def test_find_two_sum_negative_numbers():
    arr = [-1, -2, 3, 4]
    target = 2
    assert find_two_sum(arr, target) == [0, 2]


def test_find_two_sum_empty():
    assert find_two_sum([], 5) is None


def test_find_two_sum_single_element():
    assert find_two_sum([5], 5) is None

#Lab_1.py
def all_elements_equal(data: list) -> bool:
    return bool(data) and len(set(data)) == 1


def find_two_sum(arr: list[int], target: int) -> list[int] | None:
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[i] + arr[j] == target:
                return [i, j]
    return None
```

Задание 3: Реализуйте и протестируйте метод String repeat(String pattern, int repeat), который строит строку из указанного паттерна, повторённого заданное количество раз.

Код:

```
def test_repeat_zero():
```

```
    assert repeat("e", 0) == ""


def test_repeat_positive():
    assert repeat("e", 3) == "eee"
    assert repeat("ABC", 2) == "ABCABC"


def test_repeat_negative():
    with pytest.raises(ValueError):
        repeat("e", -2)


def test_repeat_none_pattern():
    with pytest.raises(TypeError):
        repeat(None, 1)


def test_repeat_empty_pattern():
    assert repeat("", 5) == ""


# task_3.py
def repeat(pattern, count):
    if pattern is None:
        raise TypeError("Шаблон должен быть строкой.")
    if count < 0:
        raise ValueError("Число должно быть неотрицательным.")
    return pattern * count
```

Результат:

Вывод: освоил приемы тестирования кода на примере использования пакета pytest.